



Amadeus Technical Design Document

Generative AI: Newsletter Generation Use Case

Date: November 29, 2024
Version 2.0

Author(s): Sebastian Caucheteux Austin Jacobson

Reviewer(s):

Contents

About this document	3
Document History	4
1. Introduction	5
1.1. Document overview	5
1.2. Goals	5
Problem statement	5
2. Requirements	6
2.1. Business requirements	6
3. Design	7
3.1. Architecture	7
3.2. Project Governance	8
3.3. Front-end	8
3.4. Backend	10
3.4.1 Flask Endpoints	10
3.4.2 Daily Newsletter Generation	11
3.4.3 Weekly Digest Generation	13
3.4.4 News Search	14
3.4.5 News Parsing (Firestore)	15
3.5. Deployment	17
3.5.1 CloudRun	17
3.5.2 Firestore: Config	17

About this document

Highlights	
Purpose	To document the GenAI Newsletter Generation for Amadeus
Intended audience	A technologist audience familiar with GenAI needs and applications.

Key assumptions	The audience has some familiarity with GenAI suite and its product offerings, and may have attempted building a model using GenAI APIs on GCP themselves.
Delivery note	Use this technical design document to deliver the solution at the end of a Gen AI engagement, and as a reference for technical teams maintaining or scaling the project in the future.

Document History

S.No	Version	Description
0.5	Initial version	Initial MVP
2.0	Version 2.0	Flask/React architecture, News Search

1. Introduction

1.1. Document overview

This Technical Design Document details the technical architecture and recommendations made by the Google Field Solution Architect team for the Amadeus team to develop and scale a Gen AI MVP solution for the Competitive Intelligence platform use case.

It documents the MVP solutions built using Gemini and details the architecture and technical implementation.

1.2. Goals

Amadeus relies heavily on staying ahead of industry trends and competitor activity. Their Competitive Intelligence (CI) team currently uses a largely manual process to curate and distribute daily and weekly news summaries to employees, demanding significant time and resources. This limits their ability to analyze trends, develop proactive insights, and scale knowledge sharing effectively.

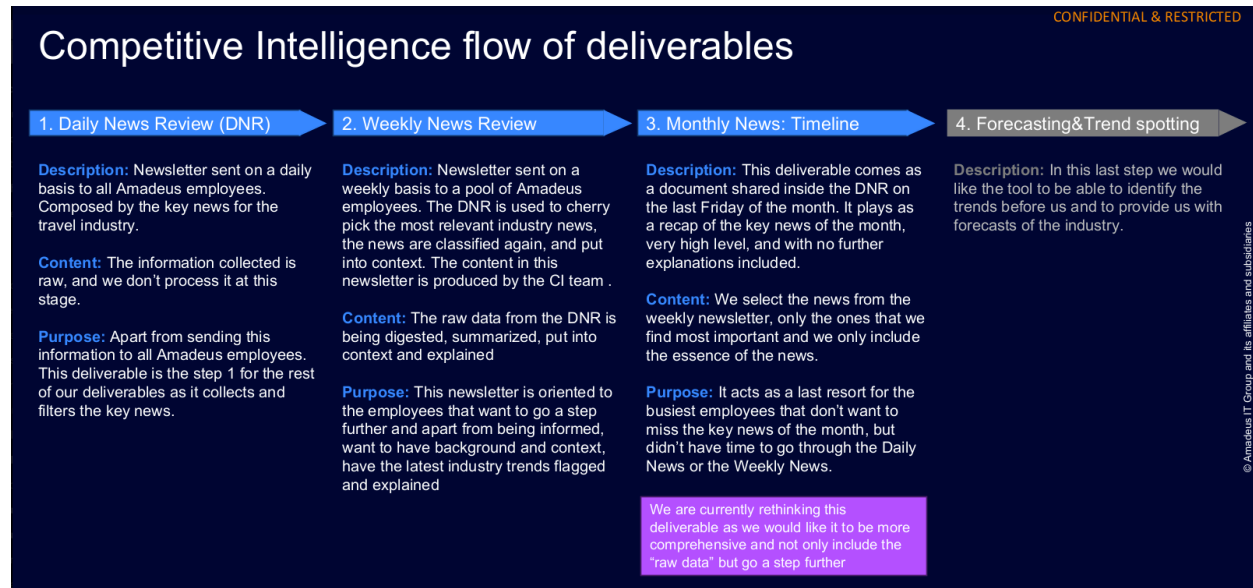
Problem statement

- **Current State:** The CI team manually reviews 65+ travel industry websites daily, creating a daily news brief and a more in-depth weekly digest. This manual approach consumes considerable staff time (estimated at 4-6 hours per day) and limits the volume of information they can process.
- **Stakeholders Impacted:** The entire Amadeus organization benefits from CI insights, particularly decision-makers in strategy, product development, sales, and marketing.
- **Ideal Future State:** Amadeus seeks to automate and enhance their CI process through a "CI Assistant" powered by Google Cloud AI. This assistant will automate news sourcing, filtering, summarization, and even trend identification, freeing the CI team to focus on analysis and strategic initiatives.
- **Anticipated ROI:**
 - Increased efficiency and staff productivity within the CI team.
 - Faster, more proactive identification of industry trends and potential disruptions.
 - Improved decision-making across Amadeus based on data-driven insights.

2. Requirements

This section lists all relevant functional and non-functional requirements.

2.1. Business requirements

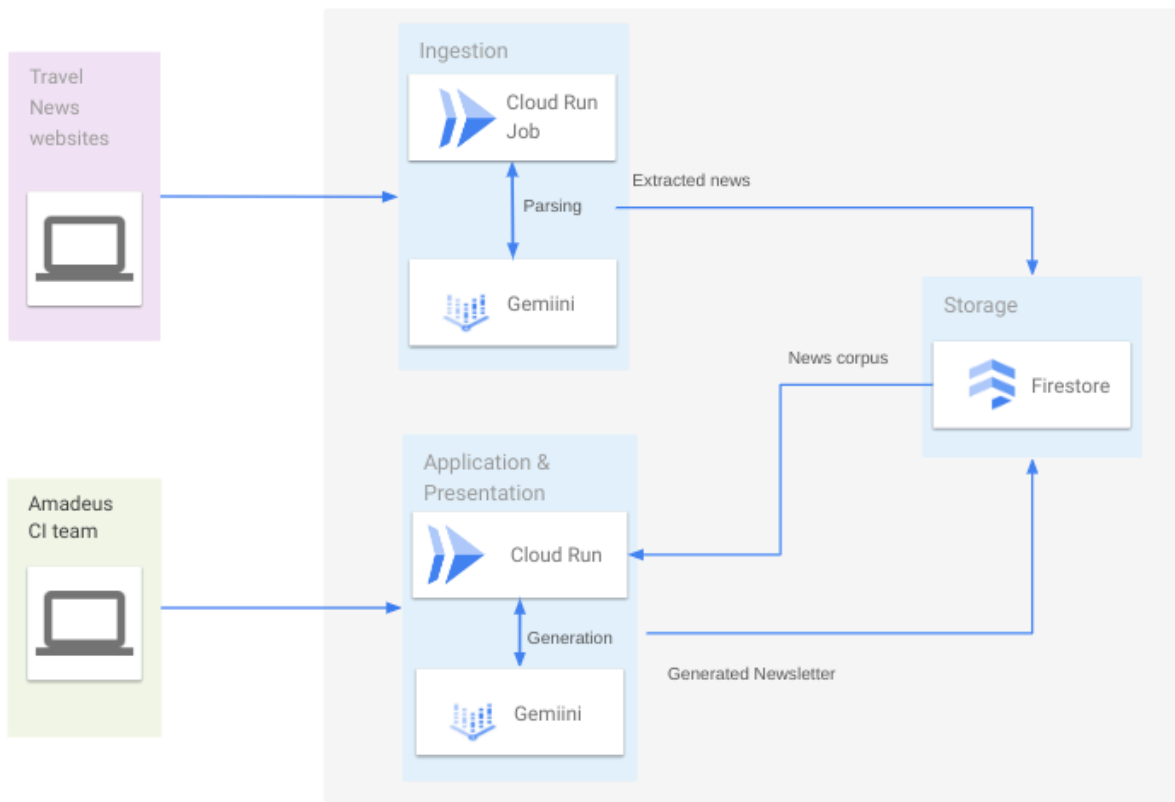


All the business requirements are listed here:

 Google CI Assistant.pptx

3. Design

3.1. Architecture



- Data Ingestion: Automated scraping of 65+ travel industry websites, with potential expansion to multimedia content (podcasts, videos). The system should handle pagination to capture all recent articles.
- AI-Powered Analysis:
 - Article Extraction: Uses Gemini for article extraction, identifying title, abstract, publication date, and URL. Implements error correction for malformed JSON outputs.
 - Newsletter Generation: Leverages Gemini Pro to structure extracted news into daily and weekly digests, including personalized insights and trend analysis.
 - Multimedia Summarization: Extracts key information from podcasts and videos, aligning with Amadeus's areas of interest (competitors, technologies, trends).
 - News Search: Search over all saved articles (not just those included in the newsletters) to identify trends, support theories, and inform arguments.
- Data Storage:

- Utilizes Firestore to store news articles, multimedia summaries, and generated newsletters, enabling historical analysis and trend identification.
 - Utilizes BigQuery to store articles for search indexing, and track articles extracted day-by-day to prevent duplication.
 - Utilizes Artifact Registry to store containers running on Cloud Run.
- User Interface: Employs a modern Flask/React architecture for the CI Platform, providing a one stop shop for daily and weekly newsletter generation, viewing, and editing, as well as news search.

3.2. Front-end

The front-end of the competitive intelligence platform is built using a modern JavaScript framework called React, specifically utilizing Vite for a faster development experience. Styling is handled by Tailwind CSS, a utility-first CSS framework that promotes rapid development and consistent design.

The front-end codebase is structured to ensure maintainability and scalability as the application grows. Key directories within the client/src folder include:

pages: The `pages` directory structures the application's main views, each represented by a separate file:

- **DailyNewsletter.tsx:** This page provides a studio for creating daily newsletters, allowing users to set date ranges, generate newsletters, and manage news items within different sections. It utilizes `NewsletterContext` and interacts with the backend via `ApiHelper` for data fetching and management.
- **SearchResults.tsx:** This page displays search results fetched from the backend using `ApiHelper`, presenting them with pagination and a summary generated by Gemini AI. It uses `useSearchParams` to retrieve the search query and dynamically updates the results.
- **WeeklyDigest.tsx:** This page offers a studio for creating weekly digests, enabling users to set start dates, generate digests, and organize news items into sections and highlights. It also uses `NewsletterContext` and `ApiHelper` for data fetching and state management.

components: This directory contains reusable UI components that are used throughout the application. These components encapsulate specific functionalities or visual elements,

promoting code reusability and consistency. Examples include `DailyNewsList.tsx`, `SearchBar.tsx`, and `Pagination.tsx`, which are used for displaying news lists, facilitating search, and navigating through paginated results.

context: The front-end utilizes React's Context API to manage application state, specifically with `NewsletterContext` which handles data related to newsletters and digests, ensuring efficient data sharing across components and avoiding prop drilling. This context provides state variables like `nlId`, `digestId`, and `flags` for new content, along with functions to update these values.

dto: The `dto` directory defines data transfer objects (DTOs) used for communication between the front-end and back-end:

- **InterfaceDefinition.tsx:** This file uses Zod to define schemas and TypeScript types for various data structures, including `News`, `Newsletter`, `Digest`, and `SearchResponse`. These schemas ensure data integrity and provide type safety.

backend: The `backend` folder contains a single file, `ApiHelper.ts`, which serves as the interface for communicating with the Flask backend. This file encapsulates all API calls using `axios`, providing functions for data retrieval, generation, saving, and deletion of newsletters and digests. It also includes functionality for searching news articles and analyzing them. Zod schemas are used to validate incoming data, ensuring data integrity.

Essentially, `ApiHelper.ts` acts as a client-side API abstraction layer, centralizing all backend interactions and promoting code organization and maintainability in the front-end.

config: The `config` directory contains a single file, `apiService.ts`, which defines the configuration for accessing the API. It defines an interface `ApiServiceConfig` to specify the structure of the configuration and uses a function `getApiUrl` to determine the appropriate API URL based on the environment (development or production). In development, it uses the `VITE_API_URL` environment variable or defaults to `http://localhost:3000/api`, while in production, it dynamically constructs the URL based on the current domain. This approach ensures that the application can seamlessly connect to the correct API endpoint in different environments.

3.3. Backend

3.4.1 Flask Endpoints

`app.py` serves as the main entry point for the Flask backend application, defining various API endpoints to handle requests related to newsletter and digest generation, news retrieval, and data management. Here's a breakdown of the endpoints grouped by their functionality:

Retrieval Endpoints

- `/api/newsletters` (GET): Retrieves newsletter history, allowing for a limit on the number of records returned.
- `/api/digests` (GET): Retrieves digest history, returning the 5 most recent digests.
- `/api/news` (GET): Retrieves news articles within a specified date range, with options for filtering by website and ranking.
- `/api/news/url/<url_hash>` (GET): Retrieves a specific news article by its URL hash.
- `/api/digest/news` (GET): Retrieves news articles for the digest generation process.
- `/api/newsletter/id` (GET): Retrieves a specific newsletter by its ID.
- `/api/digests/id` (GET): Retrieves a specific digest by its ID.
- `/api/news/search` (GET): Performs a search for news articles based on a query via the Vertex AI Search agent.

Generation Endpoints

- `/api/newsletters/generate` (POST): Generates a newsletter based on a specified date range and optional user-provided content. Complete description of the steps followed is included in the [Daily Newsletter Generation](#) section below.
- `/api/digests/generate` (POST): Generates digest content based on a specified date range. Complete description of the steps followed is included in the [Weekly Digest Generation](#) section below.
- `/api/digests/highlight/generate` (POST): Generates or regenerates highlights for a digest.
- `/api/digests/metadata` (POST): Generates metadata (context, key message) for a news item to be included in a digest.

Saving Endpoints

- `/api/newsletters` (POST): Saves a new newsletter to Firestore.
- `/api/digests` (POST): Saves a new digest to Firestore.

Update Endpoints

- `/api/newsletters/<newsletter_id>` (PUT): Updates an existing newsletter in Firestore.

- `/api/digests/<digest_id>` (PUT): Updates an existing digest in Firestore.
- `/api/articles/<article_url>` (PUT): Updates an existing article in Firestore.

Deletion Endpoints

- `/api/newsletters/<newsletter_id>` (DELETE): Deletes a newsletter from Firestore.
- `/api/digests/<digest_id>` (DELETE): Deletes a digest from Firestore.

3.4.2 Daily Newsletter Generation

The `/api/newsletters/generate` endpoint orchestrates the process of generating newsletter content by fetching website content, extracting relevant news articles, filtering them, and finally structuring them into a newsletter format. It handles optional video and article links provided by the user, incorporates news from various sources, and utilizes Gemini to ensure content novelty. Functions are defined in the server directory files `firebase_helpers.py` and `newsletter_generation.py`

1. Process User-Provided Content (Optional)

- **functions:** `request.get_json()`, `request.form.get()`, `fetch_and_analyze_media()`, `get_html_content()`, `extract_news_with_gemini()`
- **action:** Retrieves video and article links from the request, processes YouTube videos concurrently using `fetch_and_analyze_media()`, fetches HTML content for articles using `get_html_content()`, and extracts news data using `extract_news_with_gemini()`.
- **purpose:** Incorporates user-specified content into the newsletter.

2. Fetch and Pre-process News Articles

- **functions:** `datetime.fromisoformat()`, `datetime.combine()`, `get_news_by_date_range()`, `get_published_raw_news()`
- **action:** Parses start and end dates, retrieves news articles within the specified date range using `get_news_by_date_range()`, and fetches previously published news using `get_published_raw_news()`.
- **purpose:** Gathers raw news data and sets the stage for filtering and selection.

3. Identify and Prioritize News from Top Sources

- **functions:** `get_top_sources()`, `urlparse()`

- **action:** Identifies top news sources using `get_top_sources()` and extracts news articles from these sources, prioritizing them in the `top_news_dict`.
 - **purpose:** Ensures inclusion of important news from reliable sources.
4. Select Important News from Remaining Articles
 - **functions:** `select_important_news()`
 - **action:** Selects important news articles from the remaining pool using `select_important_news()` and adds them to the `top_news_dict`.
 - **purpose:** Expands the selection of top news by identifying important articles from other sources.
 5. Remove Duplicate News using Gemini:
 - **functions:** `find_duplicate_news()`
 - **action:** Utilizes Gemini to identify and remove duplicate news articles from `top_news_dict` by comparing them with previously published news.
 - **purpose:** Ensures the newsletter contains fresh and non-repetitive content.
 6. Group and Filter News by Website
 - **functions:** `group_news_by_website()`, `filter_news_by_date_range()`
 - **action:** Groups the remaining news articles by website using `group_news_by_website()` and filters them by the specified date range using `filter_news_by_date_range()`.
 - **purpose:** Organizes and prepares the news data for newsletter generation.
 7. Generate Newsletter Text
 - **functions:** `generate_newsletter_text()`, `transform_newsletter()`
 - **action:** Generates the newsletter text using `generate_newsletter_text()`, incorporating the selected news articles and considering past newsletters to maintain content diversity. Transforms the generated newsletter into a suitable format using `transform_newsletter()`.
 - **purpose:** Constructs the final newsletter content.

3.4.3 Weekly Digest Generation

Every week, Amadeus sends out a digest of the top stories from the past week's newsletters, along with choice commentary on trends and insight into the week's news. The method `digest_generation.generate_digest` orchestrates the generation of a weekly digest by processing a list of daily newsletters, extracting relevant information, and structuring it into a coherent and informative digest format. It leverages Gemini Pro for content creation and organization, ensuring the final output is both comprehensive and engaging.

1. Retrieve Newsletters

- **functions:** `get_last_week_newsletters()`
- **action:** Fetches a list of newsletters from the previous week using `get_last_week_newsletters()`, providing the raw material for the digest.
- **purpose:** Gathers the necessary data for generating the digest.

2. Parse and Categorize News

- **functions:** `parse_newsletters()`
- **action:** Processes the retrieved newsletters using `parse_newsletters()` to extract and categorize news articles by their respective websites.
- **purpose:** Organizes the news data for easier processing and content generation.

3. Generate Digest Contents

- **functions:** `generate_digest_contents()`, `generate_weekly_digest_text()`, `generate_highlights()`, `highlights_cleanup()`
- **action:** Generates the main content of the digest using `generate_digest_contents()`. This involves:
 - Generating the digest sections with `generate_weekly_digest_text()`, utilizing Gemini Pro to structure the content.
 - Generating highlights with `generate_highlights()`, again using Gemini Pro to create a compelling overview.
 - Cleaning up the highlights markdown with `highlights_cleanup()` to ensure proper formatting.
- **purpose:** Creates the core content of the digest, including categorized news and engaging highlights.

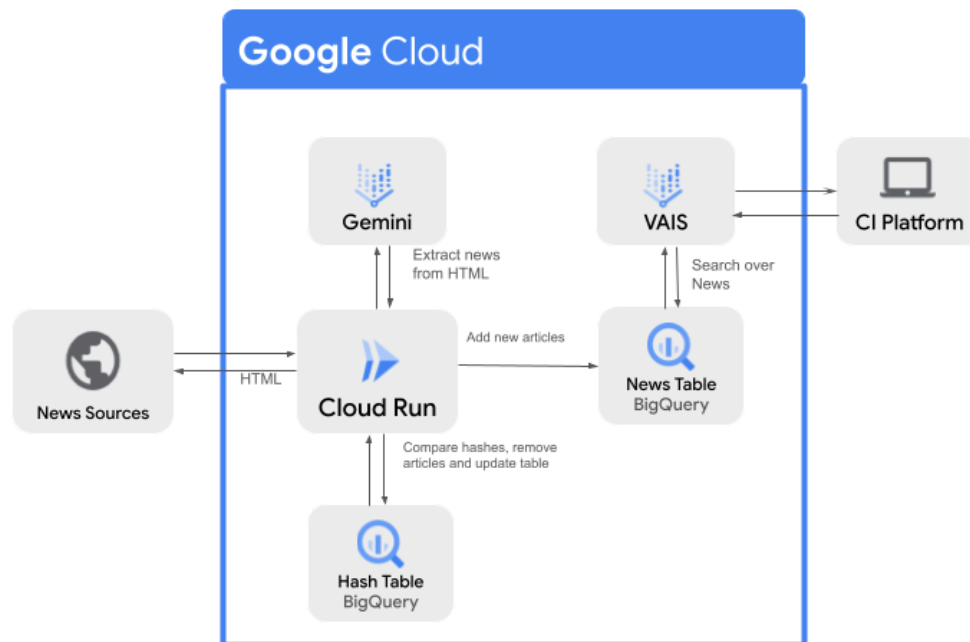
3.4.4 News Search

The Vertex AI Search solution provides a fast and reliable way for the Amadeus CI team to search over a database containing all articles scraped from their news sources dating back to July 2024, beyond just those included in the daily newsletters. This tool allows the team to investigate ongoing industry trends, quickly find stories based on theme or subject matter, and conduct thorough analyses.

News Search is built on top of a BigQuery dataset which is updated twice daily with the latest articles from a list of URLs saved in the project [configurations](#). It exists in parallel to the Firestore news collection, which is used for retrieving articles for newsletter generation, but will include the same articles. A new storage format was required due to current limitations to the Vertex AI Search Firestore database connector. Additionally, the existing BigQuery datastore

connector is enabled for automatic synchronization with a BQ table, keeping the datastore up to date without additional operational steps.

News parsing for News Search follows a similar format to parsing for Newsletter Generation, but with a few key differences. The following steps are run from a Cloud Run job, in a container generated with the Dockerfile and script saved in the engagement GitLab repo.



1. Get HTML from web sources

- **function:** `firebase_helpers.get_urls()`, `newsletter_generation.get_html_content()`, `newsletter_generation.fetch_from_url()`
- **Action:** A list of URLs is retrieved from the project [configuration Firestore collection](#). Each URL navigates to the target news source's 'front page' of articles. `get_html_content()` retrieves the full HTML contents for each page.
- **purpose:** Retrieve HTML which will be passed to Gemini for article extraction.

2. Extract article data with Gemini

- **function:** `news_fetchers.formate_and_extract()`, `newsletter_generation.extract_news_with_gemini()`
- **action:** Article data and metadata (title, url, abstract, published date) are extracted from website html. Articles are placed in a dictionary and passed to the BigQuery helper for cleaning and upload.
- **purpose:** Retrieve article data which will be stored to BigQuery

3. Clean data and upload to BigQuery

- **functions:** `bigquery_helpers.save_news_to_bigquery()`,
`bigquery_helpers.get_latest_website_hashes()`,
`bigquery_helpers.update_url_hash_rows()`
- **actions:**
 1. Compares URL hashes against the BigQuery table `url_hashes`, which contains the most recent list of hashes for each website.
 2. Articles with hashes on the list are removed, the rest are added to the `news_v2` BigQuery table.
 3. `Url_hashes` table is regenerated with the full list of hashes scraped during that run.

4. Containerize & Deploy

- **files:** `cloud_run_job/deploy_job.sh`, `cloud_run_job/main.py`,
`cloud_run_job/Dockerfile`, `cloud_run_job/requirements.txt`
- **function:** `news_fetcher.fetch_news()`
- **action:ss**
 1. Build container image with main file calling `fetch_news()` to collect news from URLs.
 2. Upload image to Artifacts Registry.
 3. Start job in Cloud Run, triggering twice daily.

Create News Search Agent

- **Engine ID:** `news-finder-v2_1730891472107`
- The datastore `news_v2_bq` indexes the BigQuery table, and is automatically synchronized using the Vertex AI Agent Builder BigQuery Structured Data datastore type.
- The Agent is then queried by the FrontEnd (`/api/news/search`), with a summary generated based on the users query and the results obtained.

3.4.5 News Parsing (Firestore)

News articles are retrieved from a list of URLs and stored to Firestore in three-times daily batches. The process is run via a CloudRun job, and the code used to generate the container image is stored in the directory `news_fetcher_v1_archive`. The job follows the steps detailed below.

1. Get HTML from web sources

- **function:** `process_v1.get_urls()`, `process_v1.get_html_content()`,
`process_v1.fetch_from_url()`

- **action:** URLs for news sites are saved to Firestore by the partner in the Config frontend page and retrieved with `get_urls`. URLs navigate to the target news source's 'front page' of articles. `get_html_content()` retrieves the full HTML contents for each page.
- **purpose:** Retrieve HTML which will be passed to Gemini for article extraction.

2. Get the datetime when last news extraction took place

News extraction is scheduled by Cloudfire on a daily basis. For each new extraction, the datetime when the previously saved 'Batch' was run is used as the low threshold for the current extraction run. Batches are saved to the 'batches' collection on Firestore, and the datetime is retrieved from there.

3. Extract articles published since the last batch from all websites

- **functions:** `process_v1.format_and_extract()`,
`process_v1.extract_news_with_gemini()`,
`process_v1.find_oldest_date()`, `process_v1.next_page_url()`,
`concurrent.futures.ThreadPoolExecutor()`,
`concurrent.futures.ThreadPoolExecutor()`
- **action:**
 - `format_and_extract` is called for each HTML obtained in step 1. Within this method, `extract_news_with_gemini` uses a prompt to obtain the data for each article listed on the page.
 - `find_oldest_date()` checks when the oldest article was published. If it is after the cutoff date obtained in step 2 (indicating the next page will contain articles within the cutoff threshold), the URL for the next page of articles is obtained with `next_page_url` and the HTML content extracted with `fetch_from_url`. The `format_and_extract` method is then called recursively with the new HTML, until the low article threshold is reached.
 - Article data for all websites listed in the Config threshold from the current batch period is returned.
- **purpose:** Scraping the target URLs provides the formatted article content used by the tool to generate Newsletters.

4. Save batch to Firestore

- **function:**
`db.collection("batches").document(batch_id).set(batch_data)`
- **action:** Save the latest article batch to Firestore.
- **purpose:** This final step ensures the article data is saved to Firestore for use by future newsletters.

3.4. Deployment

3.5.1 CloudRun

Services

The platform leverages Cloud Run to host its core microservices. Each service, including `competitive-front` and `competitive-server`, is packaged as a container and deployed independently on Cloud Run.

Jobs

CloudRun Jobs are used to execute scheduled fetching of news articles. The two jobs are packaged as containers and run independently:

- `news-fetcher-job`: The original news fetcher which collects articles based on date and stores them to Firestore. Technical details are explained in the [News Parsing \(Firestore\)](#) section below.
- `news-fetcher-job-v2`: Collects articles and saves to BigQuery for indexing and semantic search. Technical details are explained in the [News Search](#) section below.

3.5.2 Firestore: Config

Configuration values are saved to the Firestore collection 'Config', which must be manually updated from the GCP console. The configuration options are organized into two main categories:

1. Prompt Settings:

- **Daily Prompt:** This prompt defines the instructions and guidelines for the AI model when generating the daily newsletter. Users can specify the desired structure, content focus, and writing style for the newsletter.
Code: `config.get('prompt_daily', '')` retrieves the daily prompt from the configuration when [Daily Newsletter Generation](#) is executed.
- **Weekly Prompt:** This prompt provides instructions for the AI model when generating the weekly digest. Users can customize the digest's structure, content focus, and writing style.

Code: `config.get('prompt_weekly', '')` retrieves the weekly prompt from the configuration when [Weekly Digest Generation](#) is executed.

- **Weekly Highlights Prompt:** This prompt provides instructions for the AI model to select articles and generate the highlights section for the weekly digest. Users can customize the structure and selection criteria.

Code: `config.get('prompt_weekly_highlights', '')` retrieves the highlights prompt from the configuration when [Weekly Digest Generation](#) is executed, or when the highlights are updated from the front end.

2. Source Sites:

- **URL List:** The UI provides a data editor for managing the list of source websites. Users can add, remove, or edit URLs, defining the websites from which the system will collect news articles.

Code: `config.get('urls', [])` retrieves the list of URLs from the configuration when batch news fetching [jobs](#) are executed.